

Informe breve de resultados

Héctor García

Pablo Manuel Rodríguez Sosa

1. Modelo de lenguaje

Se ha entrenado desde cero un modelo de lenguaje pequeño basado en un tokenizador BPE y una arquitectura transformer causal. Los principales hiperparámetros explorados fueron el tamaño del vocabulario, la longitud de contexto, las dimensiones internas del modelo, el número de cabezas de atención y el número de capas transformer.

La configuración finalmente seleccionada fue:

- `vocab_size=95`
- `seq_len=96`
- `dim_embedding=105`
- `dim_attention=140`
- `num_heads=2`
- `num_layers=2`

Esta configuración corresponde al artefacto `artifacts/llm/best_general`.

2. Criterio de evaluación

Para comparar los distintos LLMs no se utilizó únicamente la `val_loss`. El motivo principal es que, al cambiar `vocab_size`, también cambia la tokenización. Por tanto, una pérdida por token no es directamente comparable entre modelos: el mismo texto puede quedar dividido en un número distinto de tokens según el vocabulario utilizado.

Para resolver este problema se empleó una métrica normalizada por carácter: *bits por carácter* (BPC). El cálculo puede entenderse en cuatro pasos:

1. **Loss por batch.** En primer lugar, se calcula la *cross entropy* sobre todas las predicciones evaluadas dentro de un batch. Esta es la pérdida total acumulada para ese conjunto de ejemplos.
2. **Loss por token.** Después, la pérdida total del batch se divide entre el número de tokens evaluados. Así se obtiene una pérdida media por token. Esta métrica es útil dentro de un mismo tokenizador, pero no basta para comparar modelos con distinto `vocab_size`.
3. **Loss por carácter.** Este es el paso clave. La pérdida por token se transforma a escala de carácter multiplicando por `tokens_per_char`, o equivalentemente dividiendo por `chars_per_token`. De esta forma, la métrica deja de depender directamente de cuántos tokens produce cada tokenizador para representar el mismo texto.

4. **Paso a bits por carácter.** Finalmente, como la *cross entropy* se calcula con logaritmo natural, la pérdida está expresada en *nats*. Para expresarla en bits, se divide entre $\ln(2)$. El resultado final es el BPC.

De forma resumida:

$$\text{BPC} = \frac{\text{loss}_{\text{token}} \cdot \text{tokens_per_char}}{\ln(2)}.$$

Esta normalización permite comparar de forma más justa modelos entrenados con vocabularios distintos. Un modelo con vocabulario más grande puede tener una pérdida por token distinta simplemente porque segmenta el texto de otra manera; al pasar a pérdida por carácter y después a BPC, la comparación se realiza sobre una unidad común.

Se calcularon dos variantes principales:

- **bpc_all_tokens:** evalúa todas las posiciones predichas dentro de cada ventana.
- **bpc_last_token:** evalúa únicamente el último token de cada ventana, midiendo el rendimiento cuando el modelo utiliza todo el contexto disponible.

El modelo final se eligió como una configuración de compromiso. Resulta muy competitivo en **bpc_last_token** y también queda bien posicionado en **bpc_all_tokens**.

3. Resultados del LLM

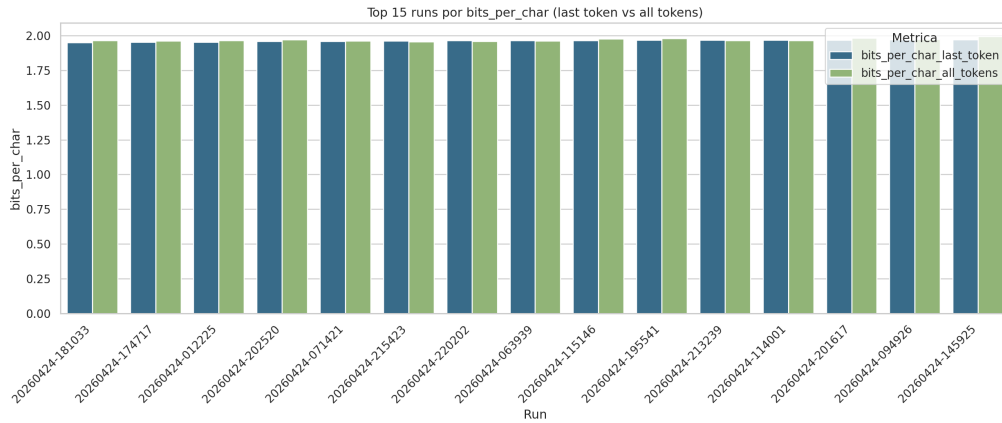


Figura 1: Comparación de los mejores experimentos del LLM con las dos métricas principales.

La comparación de los mejores experimentos muestra que no siempre coincide el mejor modelo según **bpc_all_tokens** y según **bpc_last_token**. Por ello, no se seleccionó únicamente el mínimo absoluto de una métrica, sino una configuración equilibrada entre ambas. El modelo elegido mantiene un rendimiento muy competitivo cuando se evalúa el último token, pero sin empeorar de forma clara la métrica calculada sobre todas las posiciones.

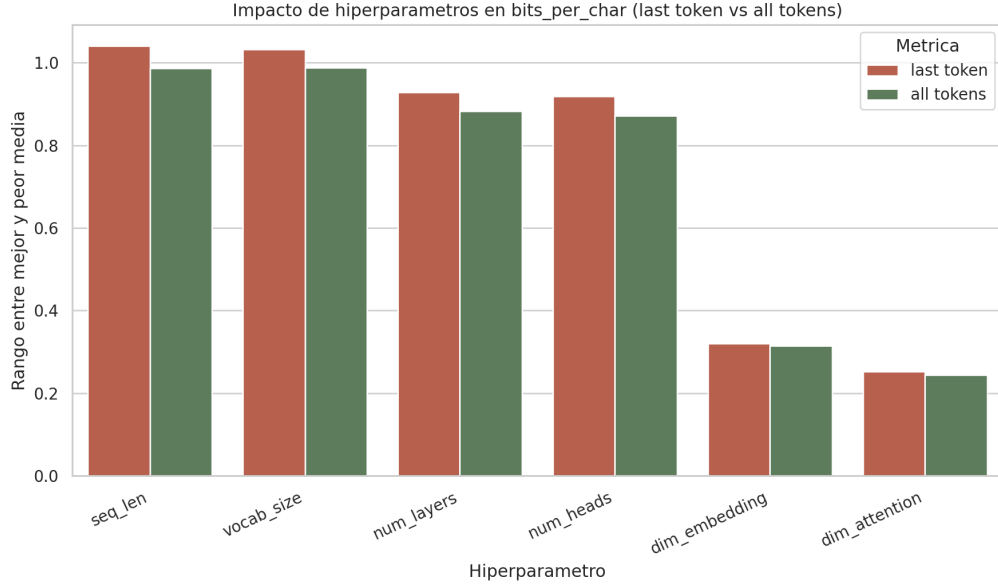


Figura 2: Impacto medio de los hiperparámetros sobre el rendimiento.

El análisis agregado por hiperparámetro indica que las variables con mayor impacto medio fueron **seq_len** y **vocab_size**. Esto es coherente con el problema: el tamaño del vocabulario modifica la granularidad de la tokenización, mientras que la longitud de contexto determina cuánta información previa puede usar el modelo para predecir el siguiente token. El número de capas y de cabezas también influye, aunque en menor medida. Las dimensiones internas presentan un efecto menos marcado en la media global.

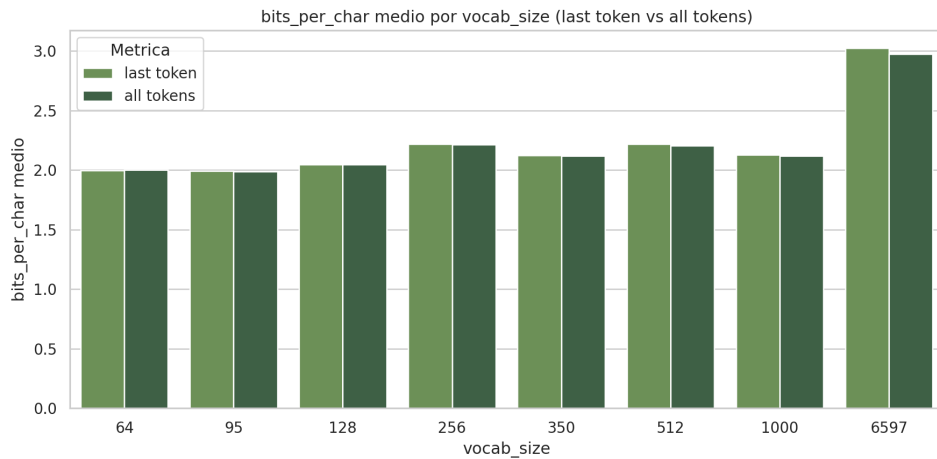


Figura 3: Bits por carácter medios en función de **vocab_size**.

La gráfica por tamaño de vocabulario muestra que los mejores resultados se concentran en vocabularios pequeños o intermedios, especialmente alrededor de **vocab_size=64** y **128**. En cambio, vocabularios demasiado grandes empeoran el rendimiento medio. Esto sugiere que, para el tamaño del corpus y del modelo utilizado, aumentar mucho el vocabulario no compensa: reduce la longitud de las secuencias, pero también incrementa el espacio de predicción y puede dificultar el aprendizaje.

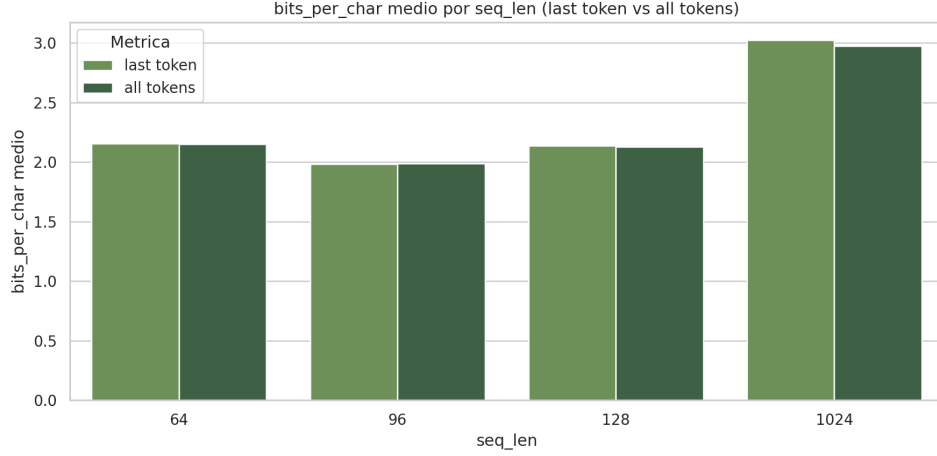


Figura 4: Bits por carácter medios en función de `seq_len`.

La longitud de contexto también tiene un efecto claro. Los mejores valores aparecen en longitudes moderadas, especialmente en torno a `seq_len=96`. Longitudes demasiado pequeñas limitan el contexto disponible, mientras que longitudes muy grandes pueden introducir secuencias más difíciles de optimizar para un modelo pequeño y con pocos datos. Por tanto, la mejor zona se encuentra en un punto intermedio.

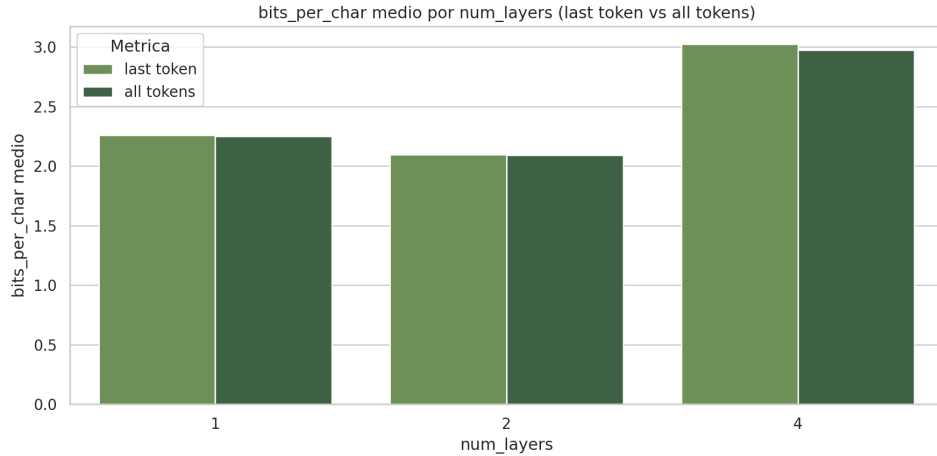


Figura 5: Bits por carácter medios en función de `num_layers`.

Respecto al número de capas, los mejores resultados se obtienen con arquitecturas poco profundas. En particular, las configuraciones con `num_layers=2` aparecen de forma consistente entre los mejores experimentos. Aumentar la profundidad no mejoró el rendimiento medio, probablemente porque el tamaño del problema no justificaba modelos más complejos o porque estos eran más difíciles de entrenar de forma estable.

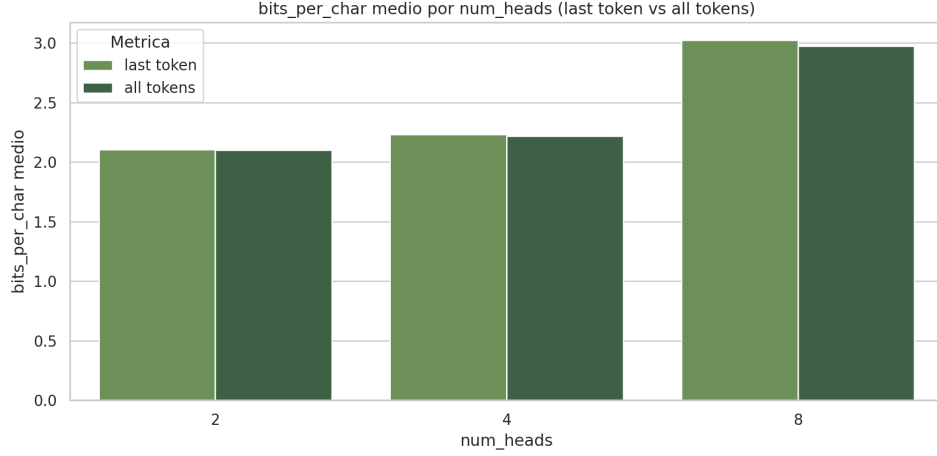


Figura 6: Bits por carácter medios en función de `num_heads`.

El comportamiento del número de cabezas sigue una tendencia similar. Las configuraciones con `num_heads=2` resultan más competitivas que alternativas con más cabezas. Esto refuerza la idea de que, en este escenario, una arquitectura compacta aprovecha mejor los datos disponibles que un modelo más grande.

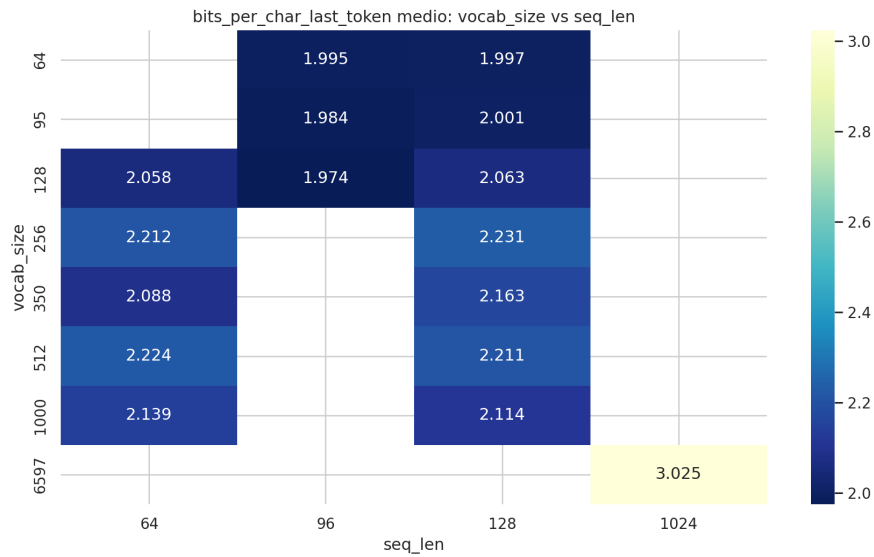


Figura 7: Interacción entre `vocab_size` y `seq_len` sobre el rendimiento medio.

Por último, el mapa de calor entre `vocab_size` y `seq_len` muestra que ambos hiperparámetros deben interpretarse conjuntamente. El tamaño del vocabulario afecta a cuántos tokens se generan para representar un texto, y la longitud de contexto fija cuántos de esos tokens puede observar el modelo. La mejor región se concentra en vocabularios pequeños o medios y longitudes de contexto moderadas, lo que coincide con la configuración final seleccionada.

En conjunto, los resultados del LLM indican que las configuraciones más grandes no fueron necesariamente mejores. La zona más competitiva del barrido corresponde a modelos compactos, con `vocab_size` reducido o intermedio, `seq_len` moderado, dos capas y dos cabezas de atención.

4. Adaptación a NER

Para la tarea de NER se reutilizó el mejor LLM como *backbone* preentrenado. Sobre esta base se añadió una cabeza de clasificación para predecir la etiqueta de cada token y se realizó un nuevo barrido de hiperparámetros centrado en `batch_size`, `epochs`, `learning_rate` y `freeze_backbone`.

En esta segunda fase se introdujeron dos cambios importantes respecto a la primera versión del sistema. En primer lugar, la función de pérdida pasó a ser una *weighted cross entropy*, con pesos calculados a partir de la frecuencia de etiquetas en entrenamiento. El objetivo era reducir el sesgo hacia la clase 0, muy dominante en el corpus, y dar más importancia a las etiquetas de entidades, mucho menos frecuentes. En segundo lugar, la selección del mejor modelo dejó de basarse en una métrica dominada por la clase mayoritaria y pasó a usar `val_entity_token_f1_micro`, calculada únicamente sobre las etiquetas de entidad y utilizada tanto para escoger el mejor checkpoint de cada entrenamiento como para actualizar el mejor artefacto global en `artifacts/ner/best`.

La mejor configuración encontrada fue:

- `freeze_backbone=False`
- `learning_rate=1e-3`
- `batch_size=8`
- `epochs=8`
- `best_val_entity_token_f1_micro` ≈ 0.7169

El mejor artefacto guardado en `artifacts/ner/best` corresponde a esta configuración. En el grid search apareció también una variante con `epochs=12` que alcanzó exactamente el mismo máximo de `val_entity_token_f1_micro`, pero sin superar el mejor valor ya registrado. Por tanto, se conservó la versión de 8 épocas como mejor candidato global. El resultado más importante es que el mejor rendimiento se obtuvo realizando *fine-tuning* completo, es decir, actualizando también los pesos del *backbone*. Esto indica que las representaciones aprendidas durante el entrenamiento del LLM son útiles como punto de partida, pero todavía necesitan adaptarse a la tarea específica de etiquetado secuencial.

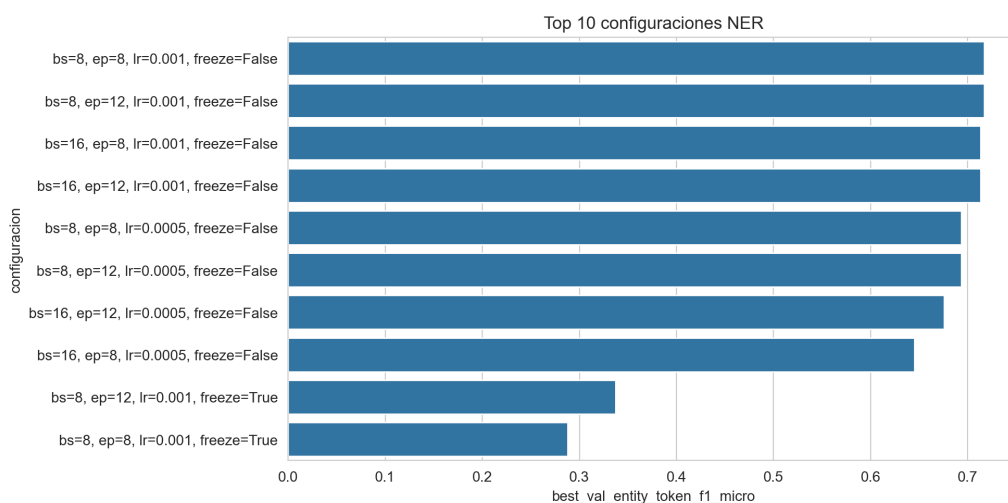


Figura 8: Mejores configuraciones evaluadas para NER.

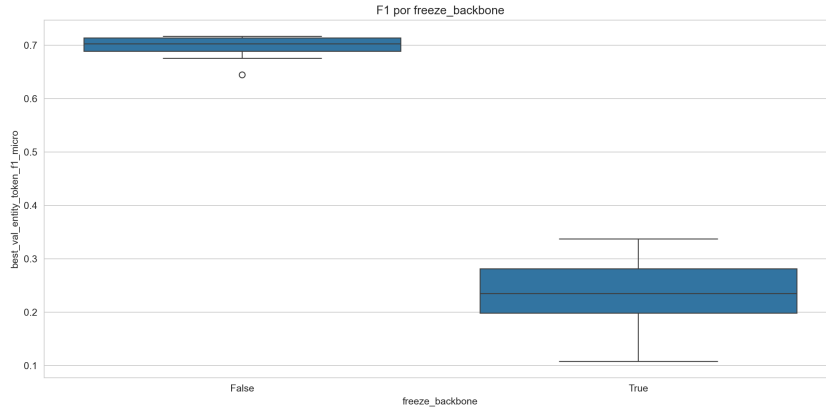


Figura 9: Distribución de `val_entity_token_f1_micro` según `freeze_backbone`.

La comparación de las mejores configuraciones muestra que los experimentos más competitivos aparecen principalmente cuando `freeze_backbone=False`. En cambio, congelar el *backbone* limita la capacidad del modelo para ajustar sus representaciones internas a las entidades del conjunto NER. Con la nueva métrica centrada en entidades esta diferencia se vuelve especialmente clara: los modelos congelados obtienen accuracies razonables por el predominio de 0, pero quedan muy por debajo cuando se evalúa solo la capacidad real de recuperar entidades. Por tanto, aunque el preentrenamiento proporciona una inicialización útil, la adaptación completa del modelo resulta clave para alcanzar el mejor rendimiento sobre entidades.

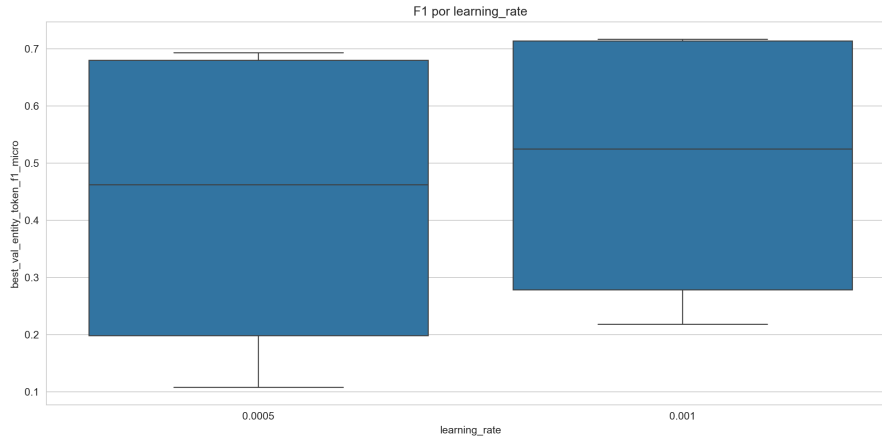


Figura 10: Distribución de `val_entity_token_f1_micro` en función del *learning rate*.

La gráfica por *learning rate* indica que $1e-3$ fue el valor más efectivo. Los valores menores tienden a producir resultados ligeramente peores, probablemente porque el ajuste del modelo avanza de forma más lenta y no aprovecha completamente las épocas disponibles. En este caso, $1e-3$ parece ofrecer un buen equilibrio entre velocidad de adaptación y estabilidad del entrenamiento.

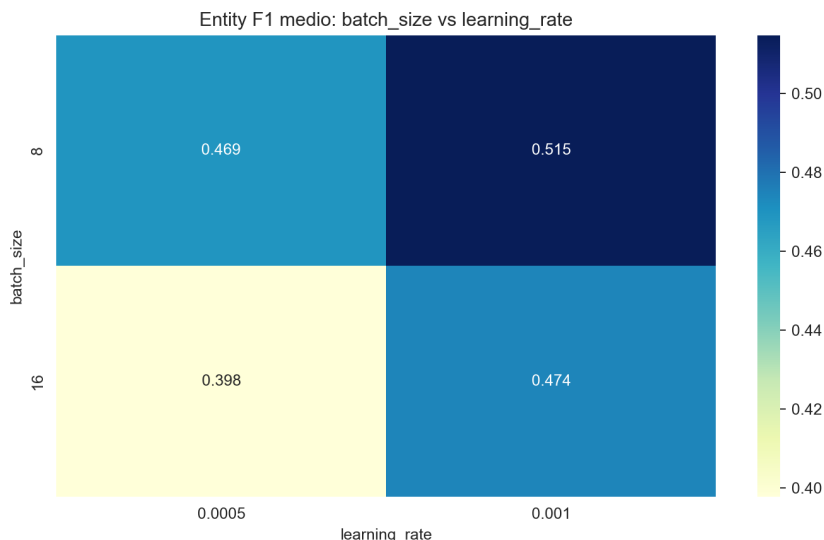


Figura 11: Relación entre `batch_size`, `learning_rate` y `val_entity_token_f1_micro`.

El mapa de calor entre `batch_size` y `learning_rate` muestra que la mejor región aparece en la combinación de `learning_rate=1e-3` con `batch_size` pequeño. En concreto, `batch_size=8` produjo el mejor valor de la métrica principal, mientras que `batch_size=16` quedó ligeramente por debajo. Además, aumentar el número de épocas de 8 a 12 no mejoró el máximo alcanzado, lo que sugiere que el modelo llega pronto a su mejor zona y que entrenar más no aporta una ventaja clara en este escenario.

En conjunto, los resultados de NER muestran que el modelo preentrenado no debe utilizarse únicamente como extractor fijo de características. El mejor comportamiento se consigue permitiendo que todo el modelo se adapte a la nueva tarea, utilizando una pérdida ponderada para compensar el desbalanceo de clases y evaluando el sistema con una métrica centrada en entidades en lugar de depender solo de la accuracy o de una métrica dominada por la clase 0.

5. Conclusiones

El barrido del LLM muestra que, al comparar modelos con tokenizadores distintos, es preferible usar métricas normalizadas por carácter en lugar de depender solo de la pérdida por token. La clave es transformar la pérdida a escala de carácter antes de expresarla en bits, ya que esto permite comparar modelos con distinto `vocab_size` sobre una unidad común.

La mejor configuración del LLM corresponde a un modelo compacto, con vocabulario pequeño, longitud de contexto moderada y una arquitectura de dos capas y dos cabezas. Los resultados indican que, para este escenario, aumentar el tamaño del modelo no aporta necesariamente una mejora y puede empeorar la optimización.

En la tarea de NER, el mejor resultado se obtuvo reutilizando el LLM preentrenado y realizando *fine-tuning* completo. Congelar el *backbone* dio peores resultados, lo que confirma que las representaciones aprendidas por el LLM son útiles, pero deben adaptarse a la tarea final. La configuración más estable combinó `learning_rate=1e-3`, `batch_size=8` y 8 épocas, alcanzando una `best_val_entity_token_f1_micro` aproximada de 0.7169. El uso de pérdida ponderada y de una métrica centrada en entidades permitió evaluar de forma más fiel el rendimiento real del sistema bajo fuerte desbalanceo de clases.

En conjunto, los experimentos muestran que un LLM pequeño, correctamente evaluado y ajusta-

do, puede servir como base eficaz para una tarea posterior de etiquetado secuencial como NER.